Docker images (Definition)

- A Docker image is a lightweight, standalone, executable package of software that includes everything needed to run an application: code, runtime, system tools, system libraries and settings.
- Images are made up of a series of layers that represent changes made to the image.
- Each layer corresponds to a set of file changes, such as adding files or modifying configurations.

Docker images (Commands)

command	description
`docker image pull`	Download an image from a registry
`docker image ls`	List images
`docker image rm`	Remove one or more images
`docker image inspect`	Display detailed information on one or more images
`docker image tag`	Create a tag for a DEST_IMAGE that refers to a SRC_IMAGE
`docker image build`	Build an image from a Dockerfile
`docker image push`	Upload an image to a registry
`docker image history`	Show the history of an image
`docker container commit`	Create a new image from container's changes

Image examples

```
# pull the ubuntu image with version 20.04 from official Docker Hub repo:
docker image pull ubuntu:20.04

# list all images exist on your host:
docker image ls

# list all information about the ubuntu image you pulled:
docker image inspect ubuntu:20.04

# create a new tag for the ubuntu image:
docker image tag ubuntu:20.04 mostafaye7ia/ubuntu:v1

# push the ubuntu image with custom tag from you host to your Docker Hub repo:
docker image push mostafaye7ia/ubuntu:v1

# remove the ubuntu tag from your host, but keep the custom tag you created previously:
docker image rm ubuntu:20.04
```

Docker images (Demo)

- Play with image commands (ls/pull/rm/inspect/history/build/push/…)
- Pull from & Push to Docker Hub with specific tags (ubuntu:20.04)
- Create a new tag for an existing image
- Create a new image from a container

Docker images (Notes)

- Gets a tag (latest) if not explicitly defined by the user.
- Can be referenced by its id or name:tag in the commands.
- Can be stored locally, in a cloud (example: ECR, ACR, GCR), a private (example: Nexus) or a public registry (example: Docker Hub).
- All containers running from an image should be removed to completely remove that image.

Dockerfile (Definition)

- A Dockerfile is a text document that contains all the commands (instructions) a user could call on the command line to assemble an image.

Dockerfile (Instructions)

instructions	description
`FROM ubuntu:20.04`	Define a base for your image.
`RUN apt update`	Execute any commands in a new layer on top of the current image and commits the result.
`WORKDIR /var/www/html/`	Set the working directory for instructions that follow it in the Dockerfile.
`COPY index.html index.html`	Copy files or directories from host to container.
`USER root`	Set current user.
`ENV KEY=VALUE`	Set environment variables..
`ENTRYPOINT ["echo"]`	Specify default executable.
`CMD ["Hello world"]`	Specify default arguments.

Dockerfile (Demo)

Let's prepare & build our first Dockerfile:

1. Use ubuntu 20.04 as a base image
2. Update the apt package manager
3. Install nginx
4. Stop nginx service
5. Set the current working directory to /etc/nginx/
6. Copy our local nginx.conf file into the container image
7. start nginx service
8. Include additional instructions: environment variables & arguments & expose ports

Dockerfile example

```
FROM golang:1.23

WORKDIR /src

COPY main.go main.go

RUN go build -o /bin/hello ./main.go

CMD ["/bin/hello"]
```

```
# main.go file content:

# package main
# import "fmt"
# func main() {
#   fmt.Println("hello, world")
# }
```

Dockerfile (Notes)

- ARG is the only instruction that may precede FROM in the Dockerfile:

```
ARG CODE_VERSION=latest
FROM base:${CODE_VERSION}
CMD /code/run-app
```

Dockerfile (Notes)

Exec form	Shell form
<ENTRYPOINT | CMD> [“command”, “param1”, “param2”]	<ENTRYPOINT | CMD> command param1 param2
PID1 is the “command”	PID1 is “/bin/sh -c”
-	Inherits environment variables from the shell
-	Supports sub-commands, piping output, chaining commands, I/O redirection, etc.
Directly traps and forwards signals like SIGINT to PID1 (”command”).	Most shells do not forward process signals to child processes (command).

Dockerfile (Notes)

ENTRYPOINT	CMD
Default command	Default arguments OR Default command+arguments if the ENTRYPOINT doesn't exist
Can be overridden by --entrypoint	Can be overridden by providing command/arguments in the docker run command

Dockerfile (Notes)

COPY	ADD
supports basic copying of files/folders into the container, from the build context or from a stage in a multi-stage build.	Supports additional features for fetching files from remote HTTPS and Git URLs, and extracting tar files automatically when adding files from the build context.

ENV	ARG
Available at build-time (all stages) and run-time	Available at build-time (current stage)
Can be overridden at run-time only by --env	Not available at run-time
Can’t be overridden in the build command	Can be overridden in the build command by --build-arg
Higher priority	Lower priority
A value is required in Dockerfile	A value is optional in Dockerfile

Dockerfile (Multi-Stage Builds)

- Multi-stage builds are mainly used to optimize the image size while keeping the Dockerfile easy to read and maintain.
- Use multiple FROM statements in your Dockerfile.
- Each FROM instruction can use a different base, and each of them begins a new stage of the build.
- You can copy artifacts from one stage to another, leaving behind everything you don't want in the final image.

Multi-Stage Builds example

```
FROM golang:1.23 AS my_stage_1

WORKDIR /src

COPY main.go main.go

RUN go build -o /bin/hello ./main.go


FROM scratch

COPY --from=my_stage_1 /bin/hello /bin/hello

CMD ["/bin/hello"]
```